



《模式识别》

第七章 线性分类器与感知机

马锦华

<https://cse.sysu.edu.cn/teacher/MaJinhua>

SUN YAT-SEN University



声明：该PPT只供非商业使用，也不可视为任何出版物。由于历史原因，许多图片尚没有标注出处，如果你知道图片的出处，欢迎告诉我们 majh8@mail.sysu.edu.cn.



课程目录（暂定）

❑	第一章	课程简介与预备知识	6学时
❑	第二章	特征提取与表示	6学时
❑	第三章	主成分分析	3学时
❑	第四章	归一化、判别分析、人脸识别	3学时
❑	第五章	EM算法与聚类	3学时
❑	第六章	贝叶斯决策理论	3学时
❑	第七章	线性分类器与感知机	3学时
❑	第八章	支持向量机	3学时
❑	第九章	神经网络、正则项和优化方法	3学时
❑	第十章	卷积神经网络及经典框架	3学时
❑	第十一章	循环神经网络	3学时
❑	第十二章	Transformer	3学时
❑	第十三章	自监督与半监督学习	3学时
❑	第十四章	开放世界模式识别	6学时



Image Classification: A core task in Computer Vision

Today:

- The image classification task
- A basic data-driven approaches to image classification
 - *Linear classifier*

Image Classification: A core task in Computer Vision

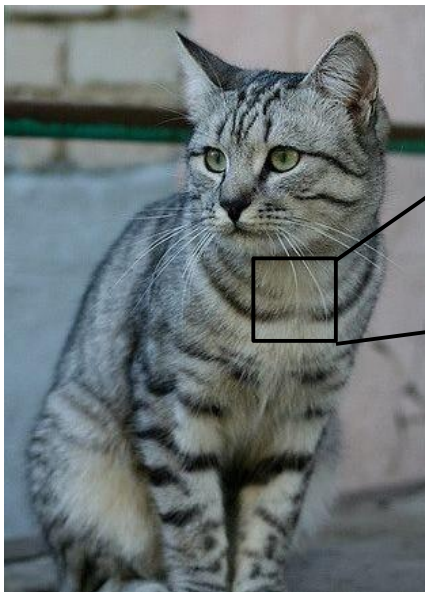


This image by [Nikita](#) is licensed under [CC-BY 2.0](#)

*(assume given a set of possible labels)
{dog, cat, truck, plane, ...}*

—————→ *cat*

The Problem: Semantic Gap



```
[105 112 108 111 104 99 106 99 96 103 112 119 104 97 93 87]
[ 91 98 102 106 104 79 98 103 99 105 123 136 110 105 94 85]
[ 76 85 90 105 128 105 87 96 95 99 115 112 106 103 99 85]
[ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]
[106 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]
[114 108 85 55 55 69 64 54 64 87 112 129 98 74 84 91]
[133 137 147 103 65 81 80 65 52 54 74 84 102 93 85 82]
[128 137 144 140 109 95 86 70 62 65 63 63 60 73 86 101]
[125 133 148 137 119 121 117 94 65 79 80 65 54 64 72 98]
[127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]
[115 114 109 123 150 148 131 118 113 109 100 92 74 65 72 78]
[ 89 93 90 97 108 147 131 118 113 114 113 109 106 95 77 80]
[ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]
[ 62 65 82 89 78 71 80 101 124 126 119 101 107 114 131 119]
[ 63 65 75 88 89 71 62 81 120 138 135 105 81 98 110 118]
[ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]
[118 97 82 86 117 123 116 66 41 51 95 93 89 95 102 107]
[164 146 112 80 82 120 124 104 76 48 45 66 88 101 102 109]
[157 170 157 120 93 86 114 132 112 97 69 55 70 82 99 94]
[130 128 134 161 139 100 109 118 121 134 114 87 65 53 69 86]
[128 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]
[123 107 96 86 83 112 153 149 122 109 104 75 80 107 112 99]
[122 121 102 80 82 86 94 117 145 148 153 102 58 78 92 107]
[122 164 148 103 71 56 78 83 93 103 119 139 102 61 69 84]]
```

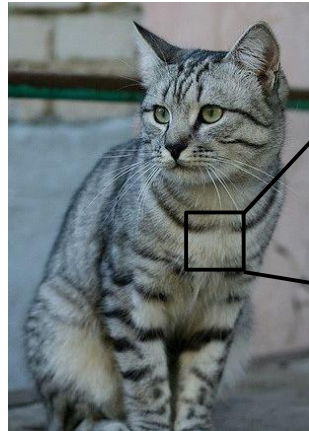
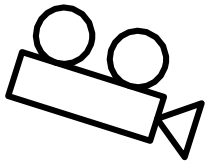
What the computer sees

*An image is a tensor of integers
between [0, 255]:*

e.g. 800 x 600 x 3 (3 channels RGB)

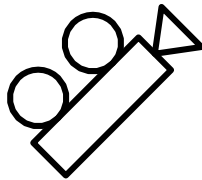
*This image by Nikita is licensed
under [CC-BY 2.0](https://creativecommons.org/licenses/by/2.0/)*

Challenges: Viewpoint variation

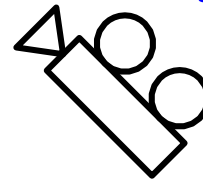


```
[185 112 100 111 104 99 106 99 96 103 112 119 104 97 93 87]
[ 91 90 102 106 104 79 98 103 99 105 123 136 110 105 94 85]
[ 76 85 90 105 120 105 87 96 95 99 115 112 106 103 99 85]
[ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]
[106 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]
[114 108 85 55 55 69 64 54 64 87 112 129 98 74 84 91]
[133 137 147 103 65 81 88 65 52 54 74 84 102 93 85 82]
[128 137 144 140 109 95 86 70 62 65 63 63 60 73 86 101]
[125 133 148 137 119 121 117 94 65 79 80 65 54 64 72 98]
[127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]
[115 114 109 123 150 148 131 118 113 109 100 92 74 65 72 78]
[ 89 93 90 97 108 147 131 118 113 114 113 109 106 95 77 80]
[ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]
[ 62 65 82 89 78 71 88 101 124 126 119 101 107 114 131 119]
[ 63 65 75 88 89 71 62 81 120 130 135 105 81 98 110 118]
[ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]
[118 97 82 86 117 123 116 66 41 51 95 83 89 95 102 107]
[164 146 112 88 82 120 124 104 76 48 45 66 88 101 102 109]
[157 170 157 120 93 86 114 132 112 97 69 55 70 82 99 94]
[130 120 124 151 139 100 109 118 121 134 114 87 65 53 69 86]
[128 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]
[123 107 96 86 83 112 153 149 122 109 104 75 80 107 112 99]
[122 121 102 80 82 86 94 117 145 140 153 102 50 70 92 107]
[122 164 148 103 71 56 70 83 93 103 119 139 102 61 69 84]
```

All pixels change when
the camera moves!



*This image by [Nikita](#) is
licensed under [CC-BY 2.0](#)*



Challenges: Illumination



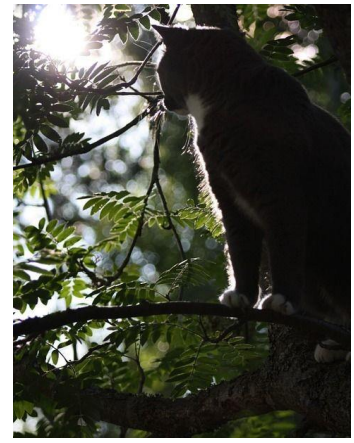
[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain

Challenges: Background Clutter



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain

Challenges: Occlusion



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain



This image by [jonsson](#) is licensed under [CC-BY 2.0](#)

Challenges: Deformation



This image by [Umberto Salvagnin](#) is licensed under [CC-BY 2.0](#)



This image by [Umberto Salvagnin](#) is licensed under [CC-BY 2.0](#)



This image by [sare bear](#) is licensed under [CC-BY 2.0](#)



This image by [Tom Thai](#) is licensed under [CC-BY 2.0](#)

Challenges: Intraclass variation



This image is [CC0 1.0](#) public domain

Challenges: Context

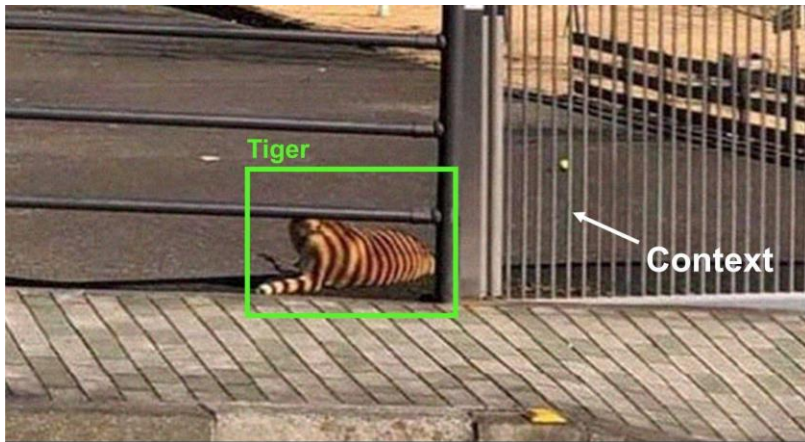
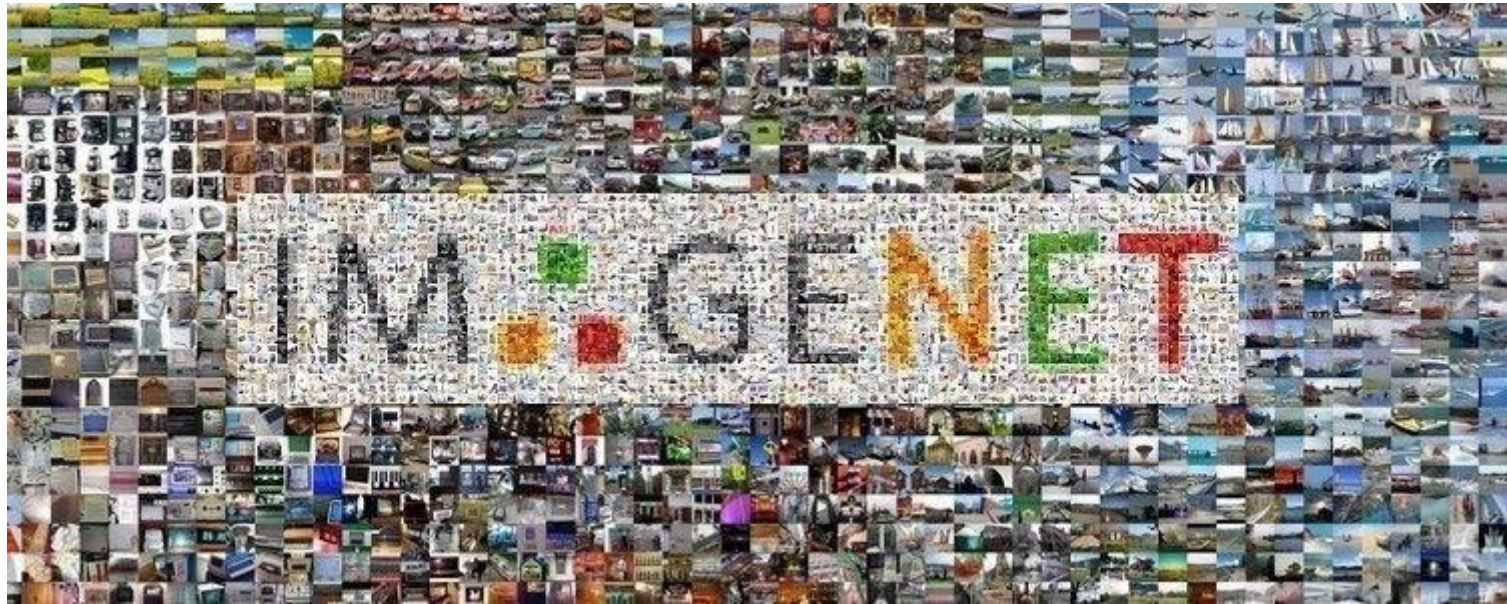


Image source:

https://www.linkedin.com/posts/ralph-aboujaoude-diaz-40838313_technology-artificialintelligence-computervision-activity-6912446088364875776-h-lq?utm_source=linkedin_share&utm_medium=member_desktop_web

Modern computer vision algorithms



This image is [CC0 1.0](#) public domain

包含1400+万张图片 and 2+万个类别。斯坦福大学李飞飞教授等人于2009年发起这个项目，并从2010年到2017年举办基于imagenet的图像分类比赛。

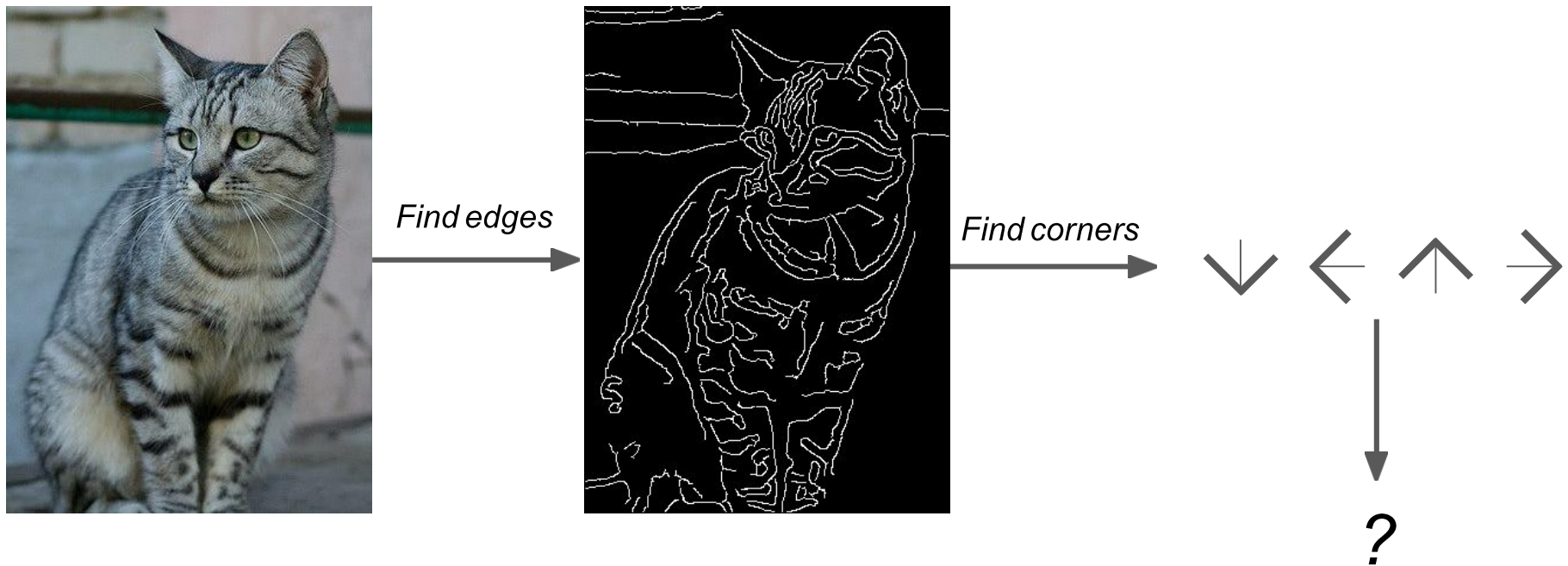
An image classifier

```
def classify_image(image):  
    # Some magic here?  
    return class_label
```

Unlike e.g. sorting a list of numbers,

no obvious way to hard-code *the algorithm for recognizing a cat, or other classes.*

Attempts have been made



John Canny, "A Computational Approach to Edge Detection", IEEE TPAMI 1986

Machine Learning: Data-Driven Approach

1. *Collect a dataset of images and labels*
2. *Use Machine Learning algorithms to train a classifier*
3. *Evaluate the classifier on new images*

Example training set

```
def train(images, labels):  
    # Machine learning!  
    return model
```

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```

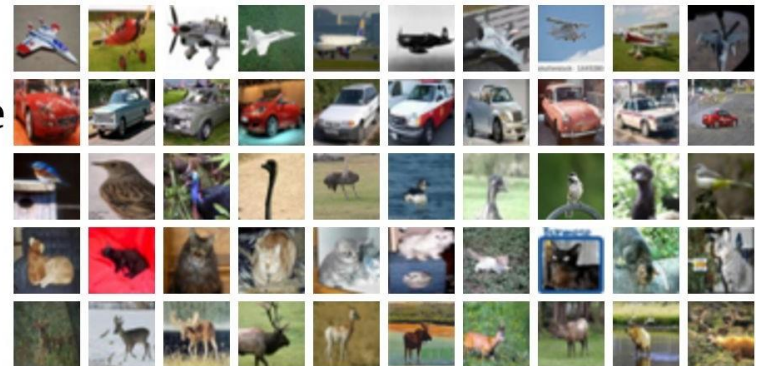
airplane

automobile

bird

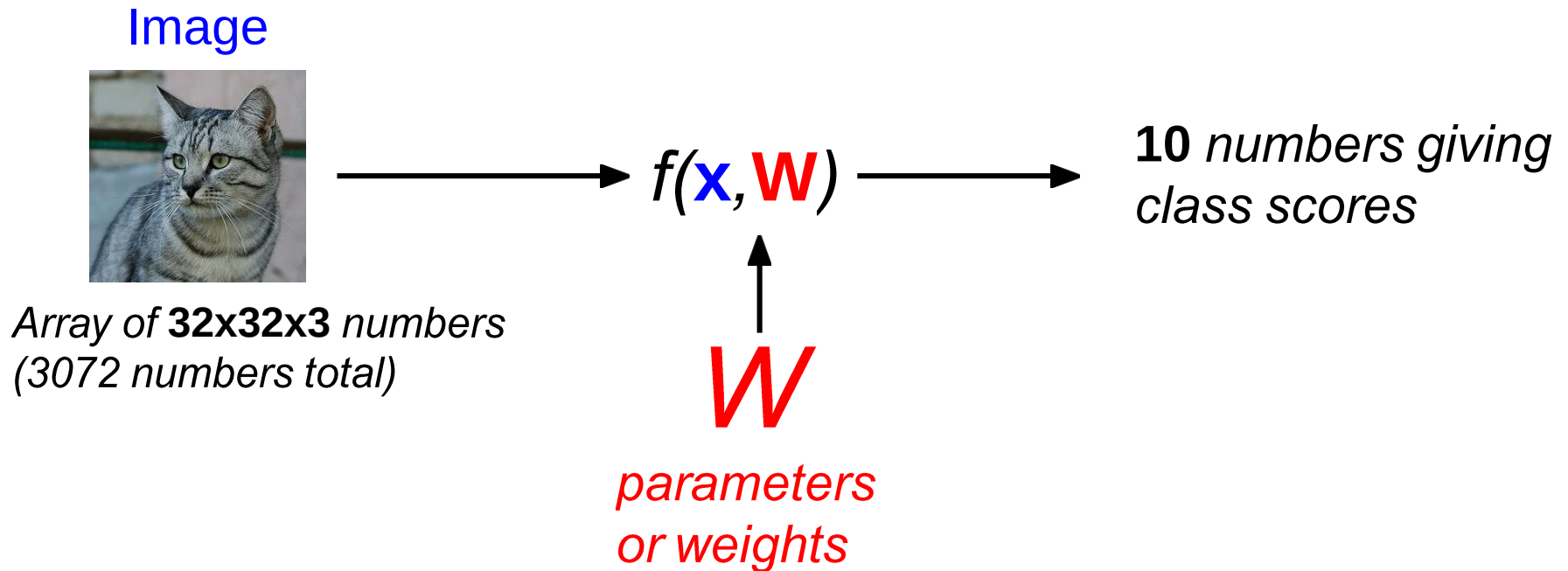
cat

deer



Linear Classifier

Parametric Approach



Parametric Approach: Linear Classifier

$$f(x, W) = Wx$$

Image



Array of **32x32x3** numbers
(3072 numbers total)



$f(\mathbf{x}, \mathbf{W})$



$\mathbf{W}$

*parameters
or weights*

10 numbers giving
class scores

Parametric Approach: Linear Classifier

$$\boxed{f(x, W)} = \boxed{W} \boxed{x}$$

10x1 10x3072 3072x1



Array of **32x32x3** numbers
(3072 numbers total)



10 numbers giving
class scores

↑
W

*parameters
or weights*

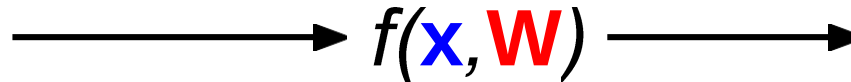
Parametric Approach: Linear Classifier

$$\boxed{f(x, W)} = \boxed{W} \boxed{x} + \boxed{b}$$

10x1 10x3072 3072x1 10x1



Array of **32x32x3** numbers
(3072 numbers total)



10 numbers giving
class scores

$\mathbf{W}$

*parameters
or weights*

Parametric Approach: Linear Classifier

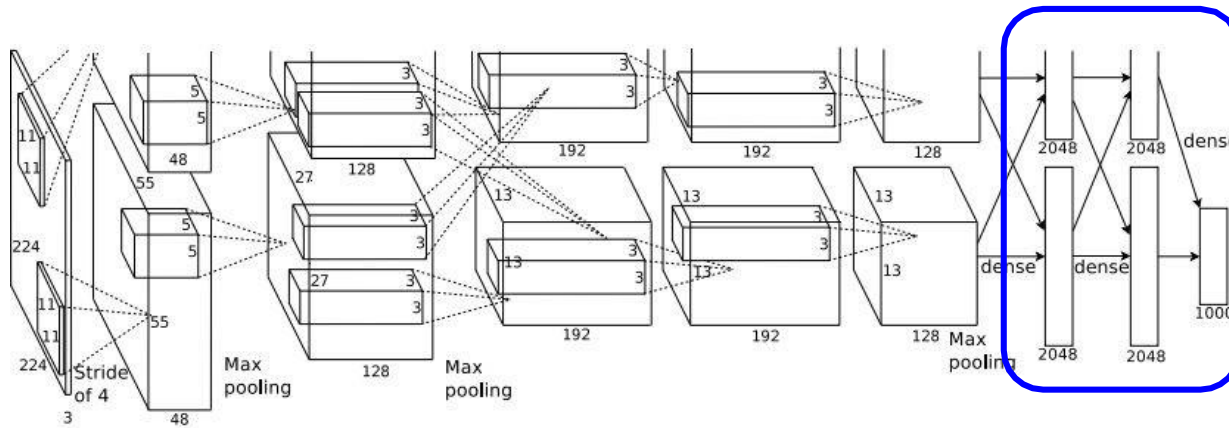
Neural Network

*Linear
classifiers*



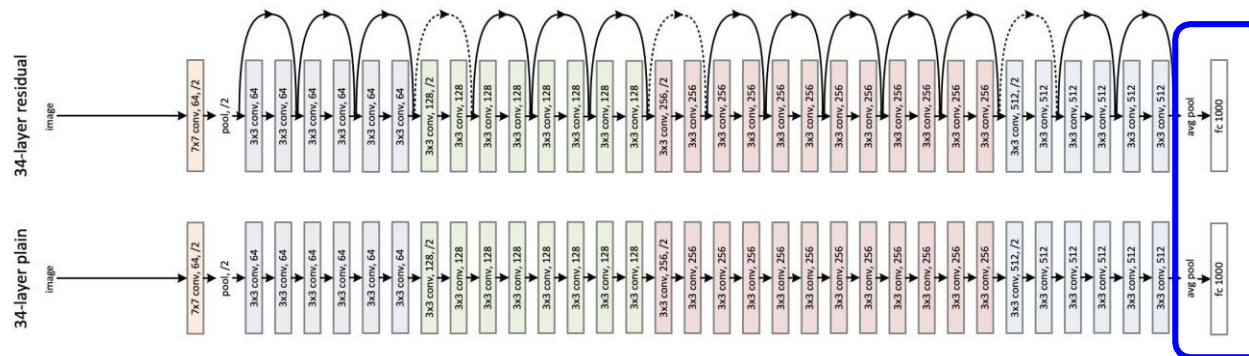
This image is [CC0 1.0](#) public domain

Parametric Approach: Linear Classifier



[Krizhevsky et al. 2012]

Linear layers



[He et al. 2015]

Recall CIFAR10

airplane

automobile

bird

cat

deer

dog

frog

horse

ship

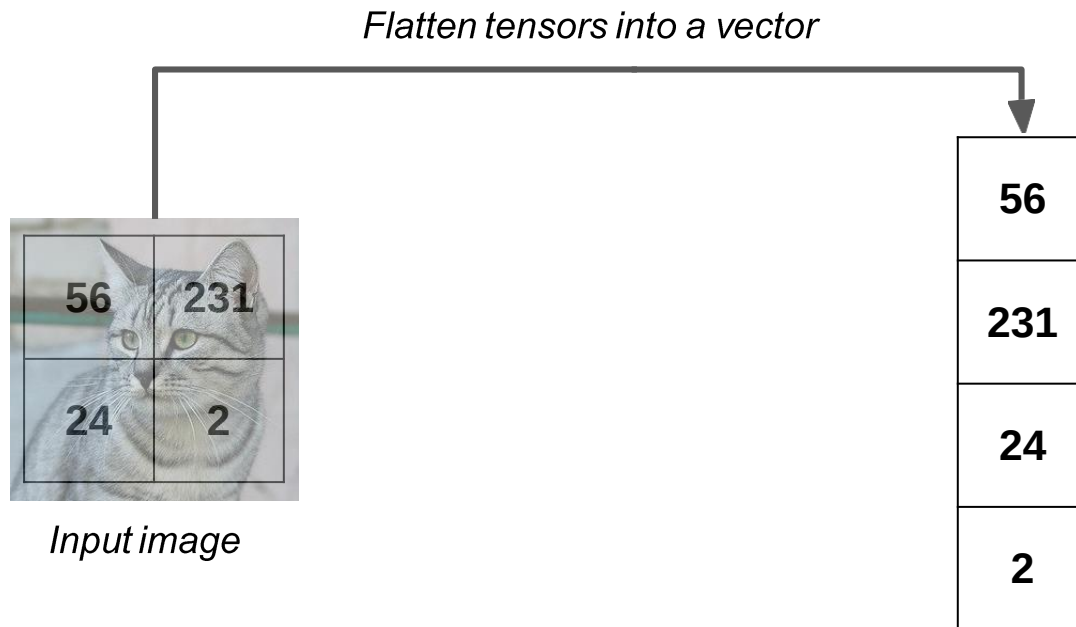
truck



50,000 *training images*
each image is **32x32x3**

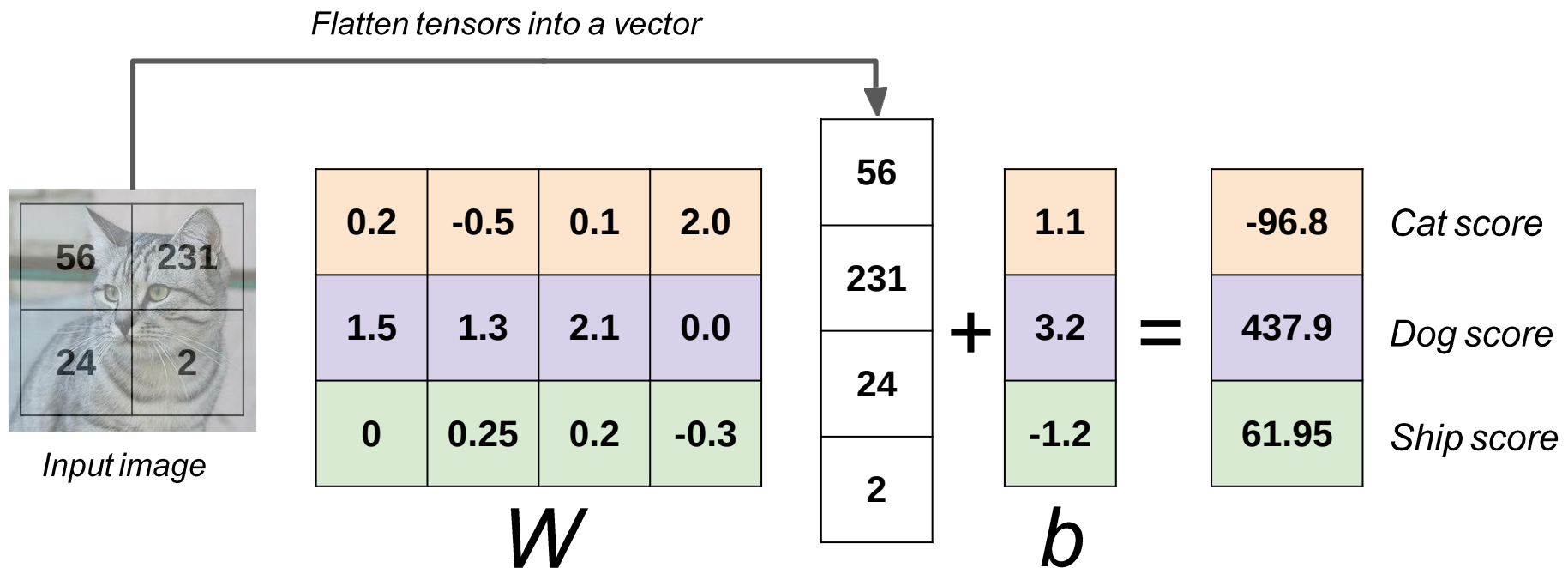
10,000 *test images.*

Example with an image with 4 pixels, and 3 classes

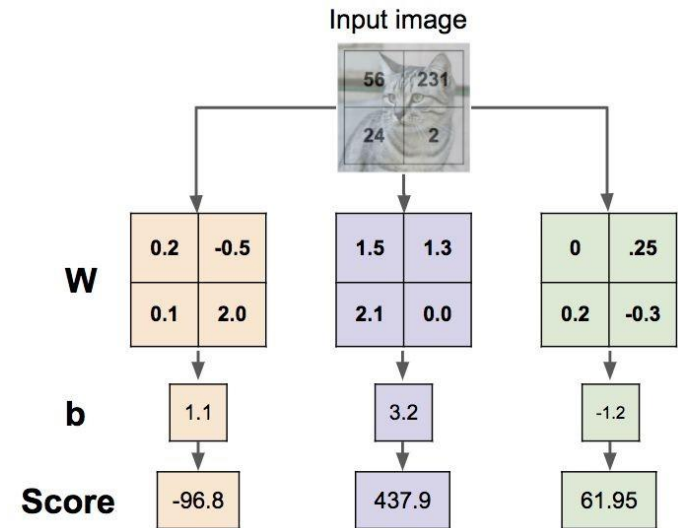
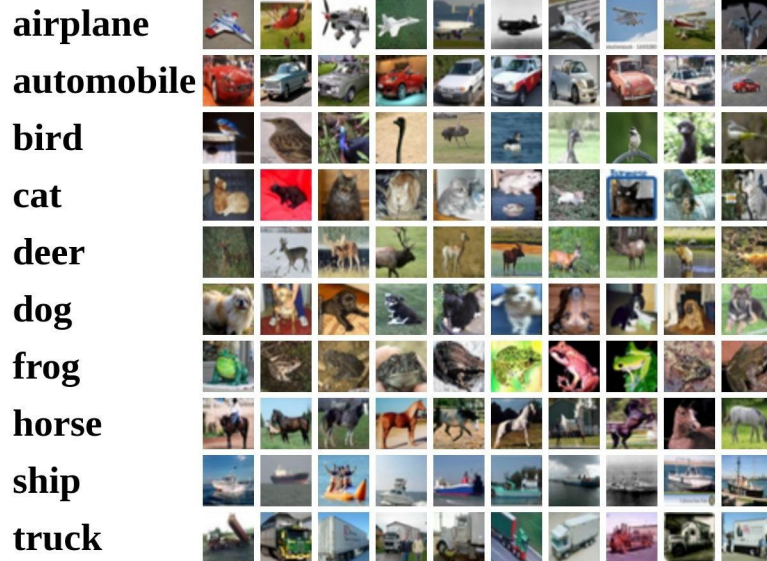


Example with an image with 4 pixels, and 3 classes

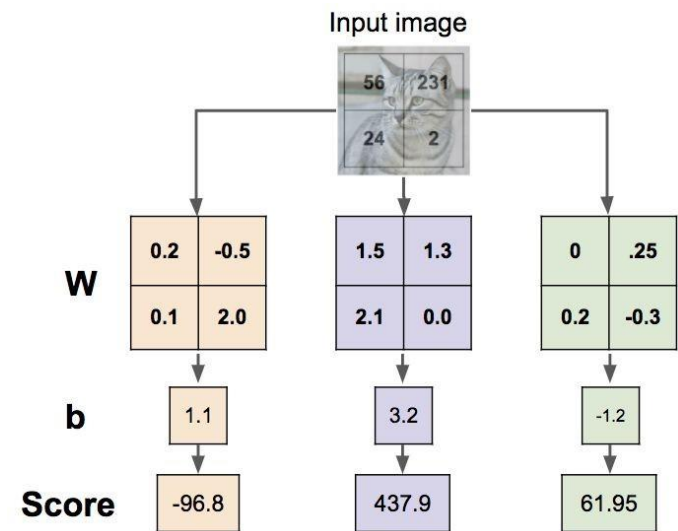
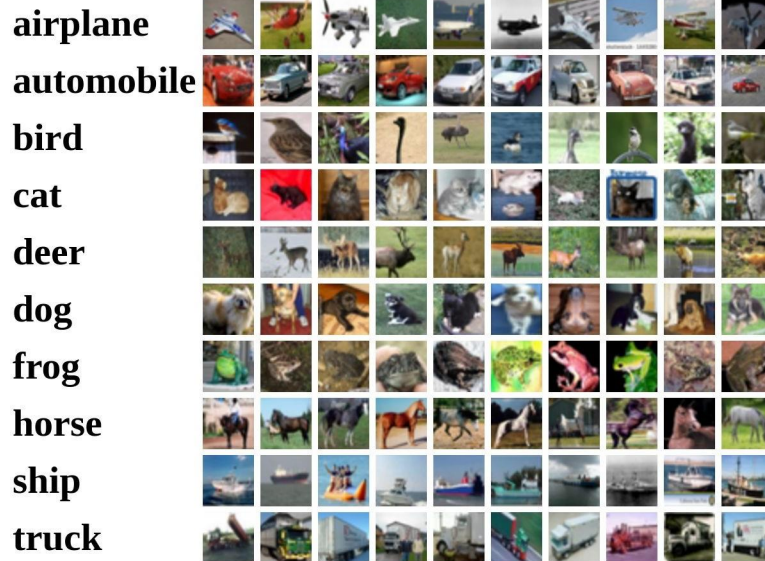
- Algebraic Viewpoint



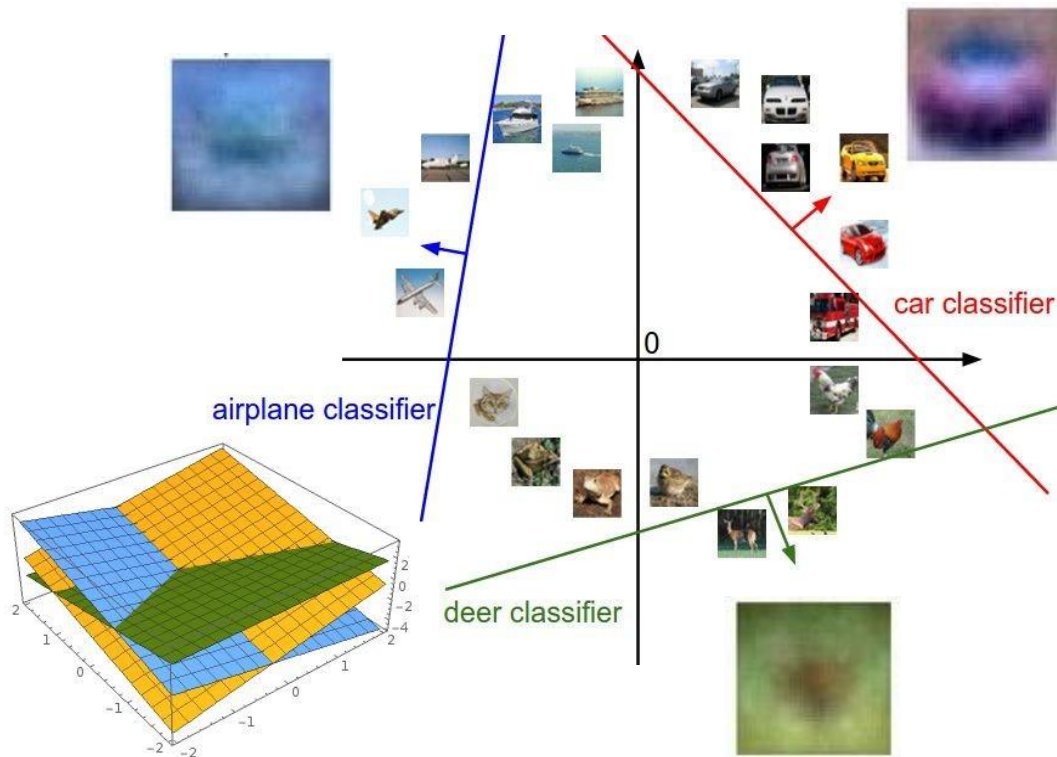
Interpreting a Linear Classifier



Interpreting a Linear Classifier: Visual Viewpoint



Interpreting a Linear Classifier: Geometric Viewpoint



Plot created using [Wolfram Cloud](#)

$$f(x, W) = Wx + b$$



Array of **32x32x3** numbers
(3072 numbers total)

[Cat image](#) by [Nikita](#) is licensed under [CC-BY 2.0](#)

Linear Classifier – Choose a good W



airplane	-3.45	-0.51	3.42
automobile	-8.87	6.04	4.64
bird	0.09	5.31	2.65
cat	2.9	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	-4.34
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

TODO:

1. Define a **loss function** that quantifies our unhappiness with the scores across the training data.
2. Come up with a way of efficiently finding the parameters that minimize the loss function. (**optimization**)

[Cat image](#) by [Nikita](#) is licensed under [CC-BY 2.0](#); [Car image](#) is [CC0 1.0](#) public domain; [Frog image](#) is in the public domain

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:

A loss function *tells how good our current classifier is*



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1

*A **loss function** tells how good our current classifier is*

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^N$$

Where x_i is image and y_i is (integer) label

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1

A **loss function** tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^N$$

Where x_i is image and y_i is (integer) label

Loss over the dataset is a average of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is image and
where y_i is (integer) label

and using the shorthand for
the scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_j + 1 \\ s_j - s_{y_i} + 1 & \text{otherwise} \end{cases}$$

$$= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Linear Classifier – Choose a good W

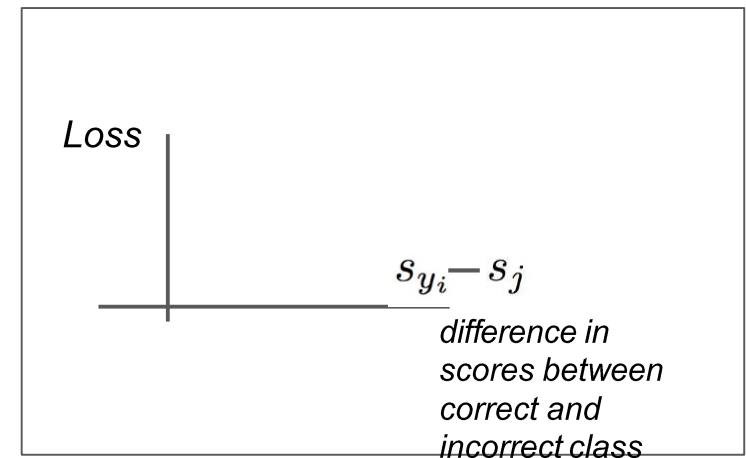
Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1

Interpreting Multiclass SVM loss:



$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_j + 1 \\ s_j - s_{y_i} + 1 & \text{otherwise} \end{cases}$$

$$= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Linear Classifier – Choose a good W

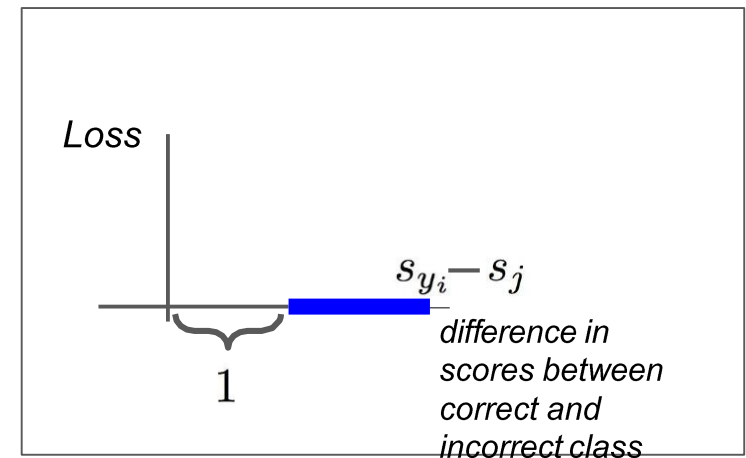
Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1

Interpreting Multiclass SVM loss:



$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_j + 1 \\ s_j - s_{y_i} + 1 & \text{otherwise} \end{cases}$$

$$= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Linear Classifier – Choose a good W

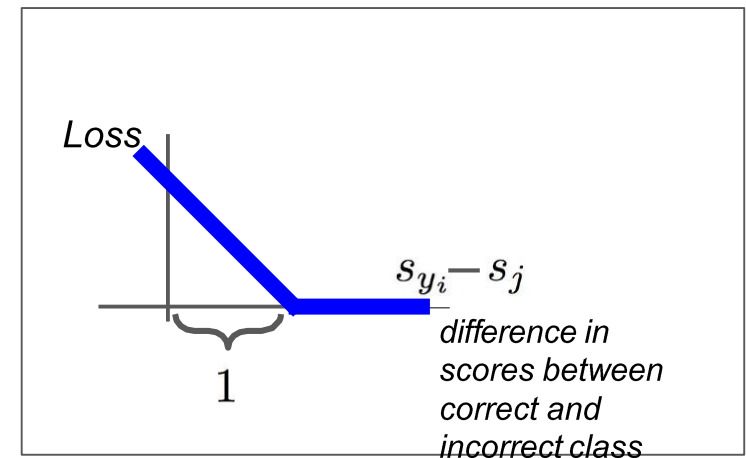
Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1

Interpreting Multiclass SVM loss:



$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_j + 1 \\ s_j - s_{y_i} + 1 & \text{otherwise} \end{cases}$$

$$= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is image and
where y_i is (integer) label

and using the shorthand for
the scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

Losses: **2.9**

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is image and
where y_i is (integer) label

and using the shorthand for
the scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$\begin{aligned}
 L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\
 &= \max(0, 5.1 - 3.2 + 1) \\
 &\quad + \max(0, -1.7 - 3.2 + 1) \\
 &= \max(0, 2.9) + \max(0, -3.9) \\
 &= 2.9 + 0 \\
 &= 2.9
 \end{aligned}$$

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

Losses:

0

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is image and
where y_i is (integer) label

and using the shorthand for
the scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$\begin{aligned}
 L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\
 &= \max(0, 1.3 - 4.9 + 1) \\
 &\quad + \max(0, 2.0 - 4.9 + 1) \\
 &= \max(0, -2.6) + \max(0, -1.9) \\
 &= 0 + 0 \\
 &= 0
 \end{aligned}$$

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is image and
where y_i is (integer) label

and using the shorthand for
the scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$\begin{aligned}
 L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\
 &= \max(0, 2.2 - (-3.1) + 1) \\
 &\quad + \max(0, 2.5 - (-3.1) + 1) \\
 &= \max(0, 6.3) + \max(0, 6.6) \\
 &= 6.3 + 6.6 \\
 &= 12.9
 \end{aligned}$$

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is image and
where y_i is (integer) label

and using the shorthand for
the scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Loss over full dataset is
average: $L = \frac{1}{N} \sum_{i=1}^N L_i$

$$L = (2.9 + 0 + 12.9)/3 = 5.27$$

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



cat 1.3

car 4.9

frog 2.0

Losses: 0

Multiclass SVM loss:

Q1: What happens to loss if car scores decrease by 0.5 for this training example?

Q2: what is the min/max possible SVM loss L_i ?

Q3: At initialization W is small so all $s \approx 0$. What is the loss L_i , assuming N examples and C classes?

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is image and
where y_i is (integer) label

and using the shorthand for
the scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Q4: What if the sum was over
all classes? (including $j = y_i$)

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is image and
where y_i is (integer) label

and using the shorthand for
the scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Q5: What if we used mean
instead of sum?

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is image and
where y_i is (integer) label

and using the shorthand for
the scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Q6: What if we used

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)^2$$

Linear Classifier – Choose a good W

Suppose: 3 training examples, 3 classes.

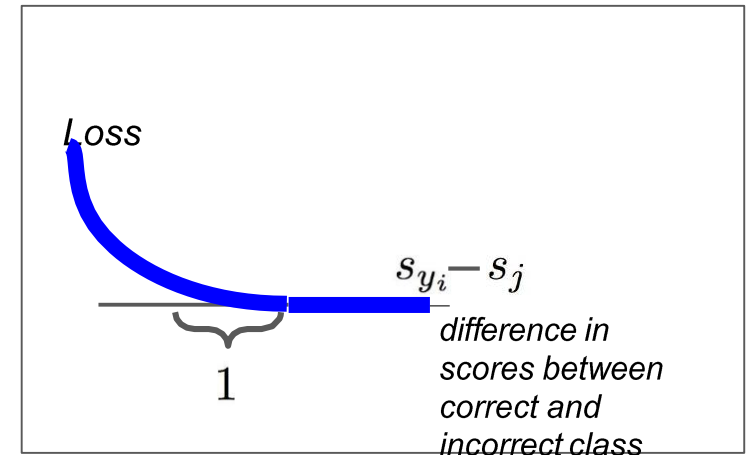
With some W the scores $f(x, W) = Wx$ are:

Multiclass SVM loss:



<i>cat</i>	3.2	1.3	2.2
<i>car</i>	5.1	4.9	2.5
<i>frog</i>	-1.7	2.0	-3.1

Losses: **2.9** **0** **12.9**



Q6: What if we used

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)^2$$

Multiclass SVM Loss: Example code

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

```
def L_i_vectorized(x, y, W):  
    scores = W.dot(x) # First calculate scores  
    margins = np.maximum(0, scores - scores[y] + 1) # Then calculate the margins s_j - s_{y_i} + 1  
    margins[y] = 0 # only sum j is not y_i, so when j = y_i, set to zero.  
    loss_i = np.sum(margins) # sum across all j  
    return loss_i
```

Softmax Classifier



Softmax Classifier (Multinomial Logistic Regression)

*Want to interpret raw classifier scores as **probabilities***



<i>cat</i>	3.2
<i>car</i>	5.1
<i>frog</i>	-1.7



Softmax Classifier (Multinomial Logistic Regression)

*Want to interpret raw classifier scores as **probabilities***



$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

<i>cat</i>	3.2
<i>car</i>	5.1
<i>frog</i>	-1.7

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

*Probabilities
must be ≥ 0*

cat	3.2	$\xrightarrow{\text{exp}}$	24.5
car	5.1		164.0
frog	-1.7		0.18

*unnormalized
probabilities*

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



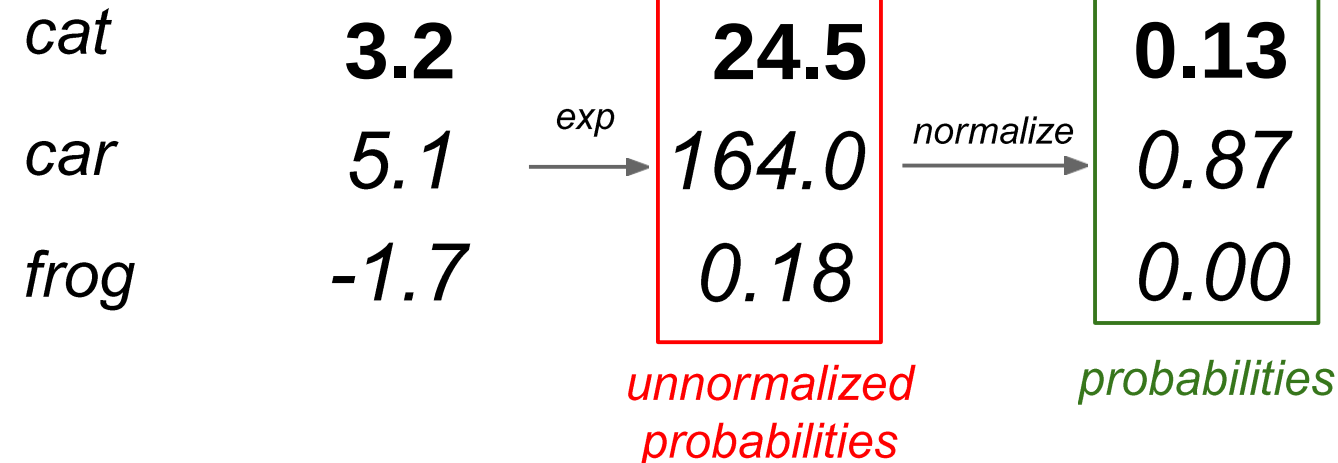
$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

*Probabilities
must be ≥ 0*

*Probabilities
must sum to 1*



Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**

$$s = f(x_i; W)$$

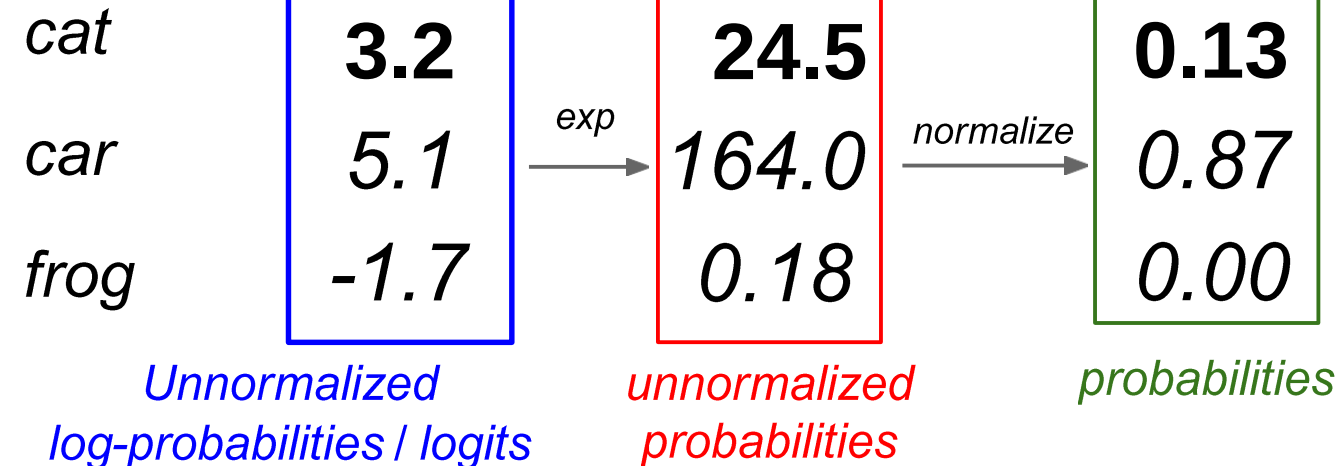
$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function



*Probabilities
must be ≥ 0*

*Probabilities
must sum to 1*



Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

*Probabilities
must be ≥ 0*

*Probabilities
must sum to 1*

$$L_i = -\log P(Y = y_i|X = x_i)$$

cat	3.2
car	5.1
frog	-1.7

Unnormalized
log-probabilities / logits

exp

24.5
164.0
0.18

unnormalized
probabilities

normalize

0.13
0.87
0.00

probabilities

$$L_i = -\log(0.13) = 2.04$$

Maximum Likelihood Estimation
Choose weights to maximize the
likelihood of the observed data

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



$$s = f(x_i; W)$$

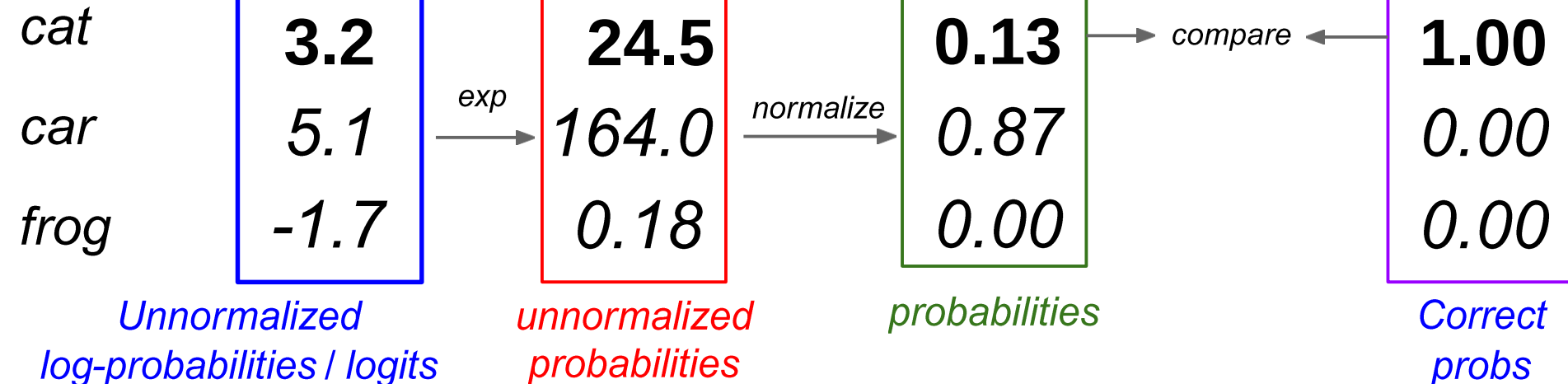
$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

*Probabilities
must be ≥ 0*

*Probabilities
must sum to 1*

$$L_i = -\log P(Y = y_i|X = x_i)$$



如何比较两个概率分布？

● 信息论！ Information theory

- 描述一个随机变量需要多少信息？
 - 假设用bit来作为信息的单位
 - 若离散变量满足 $P(x = 2) = 1, P(x \neq 2) = 0$?
 - 若离散变量是 $\{1, 2, 3, 4\}$ 上的均匀uniform分布？
- 熵entropy
 - $H = -\sum_{i=1}^m P_i \log_2 P_i$ (m 个离散可能取值，各为 P_i)
 - 如果 $P_i = 0$?
 - 定义 $0 \log_2 0 = 0!$ ，因为 $\lim_{x \rightarrow 0} x \log_2 x = 0$
 - 什么时候最大？ 什么时候最小？
 - 均匀分布的时候最大， $\log_2 m$ (琴生不等式)
 - 单点分布最小，0

Differential entropy

- 如果分布是连续的？

$$h(x) = -\int p(x) \ln(p(x)) dx$$

- 自然对数，单位是nat（奈特）
- 若 $X \sim N(\mu, \sigma^2)$ ，则 $h(X) = \frac{1}{2} \ln(2\pi e \sigma^2)$ nats
- 在所有均值和方差固定的连续分布中，高斯分布具有最大的熵
 - 或者说，不确定性uncertainty最大

Joint, conditional entropy

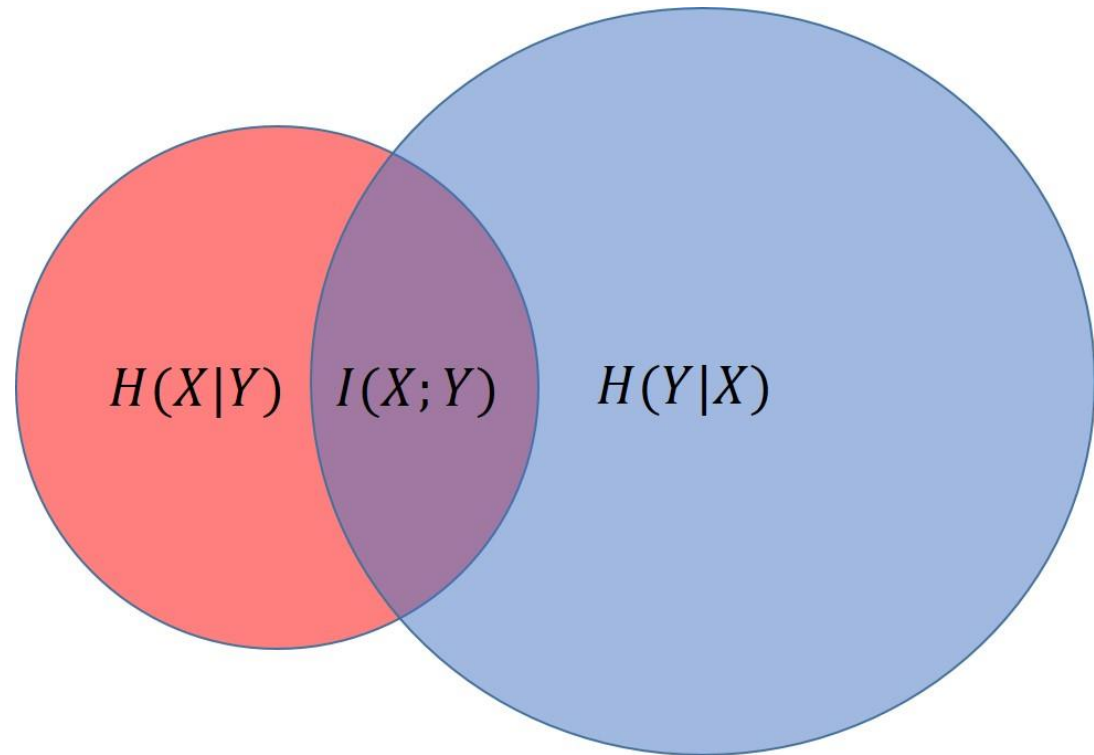
- $H(X, Y) = -\sum_x \sum_y P(x, y) \log_2 P(x, y)$
- $h(X, Y) = -\int p(x, y) \ln p(x, y) dx dy$
- $H(X|Y) = \sum_y p(y) H(X|Y = y) = \sum_{x,y} P(x, y) \log_2 \frac{P(y)}{P(x, y)} =$
 $-\sum_{x,y} P(x, y) \log_2 \frac{P(x, y)}{P(y)}$
- $h(X|Y) = -\int p(x, y) \ln p(x|y) dx dy = -\int p(x, y) \ln \frac{p(x, y)}{p(y)} dx dy$

各种熵之间的关系

- $H(X, Y) = H(Y) + H(X|Y)$
- $H(X, Y) = H(X) + H(Y|X)$
- $H(X|Y) \leq H(X)$
- $H(Y|X) \leq H(Y)$

问题

- $H(X|Y) = H(Y|X)$?
- 那么 $I(X; Y)$ 代表什么?



互信息Mutual information

- 如果 X 和 Y 互相独立，即 $p(x, y) = p(x)p(y)$ ，或者 $P(x, y) = P(x)P(y)$
 - 上面的图应该怎么画？
- $I(X; Y)$ 表示 X 和 Y 共同的那部分信息
$$I(X; Y) = H(X) - H(X|Y) = \sum_{x,y} p(x, y) \log_2 \frac{p(x, y)}{p(x)p(y)}$$
$$= H(Y) - H(Y|X)$$
- $I(X; Y) = I(Y; X)$?
- 可以粗略的看成相似程度或者相关程度

KL散度

- Kullback-Leibler divergence: 两个离散分布 P 和 Q

$$\text{KL}(P||Q) = \sum_i P_i \log_2 \frac{P_i}{Q_i}$$

- $\text{KL}(P||Q) \geq 0$, 等号当且仅当 $\forall i, P_i = Q_i$ 时成立
- $I(X; Y) = D_{KL}(p(x, y)||p(x)p(y))$
- 可以粗略看成“距离”
- 但是, KL散度对称吗?
- 交叉熵: $\text{CE}(P, Q) = -\sum_i P_i \log_2 Q_i$
 - $\text{CE}(P, Q) = H(P) + \text{KL}(P||Q)$

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



$$s = f(x_i; W)$$

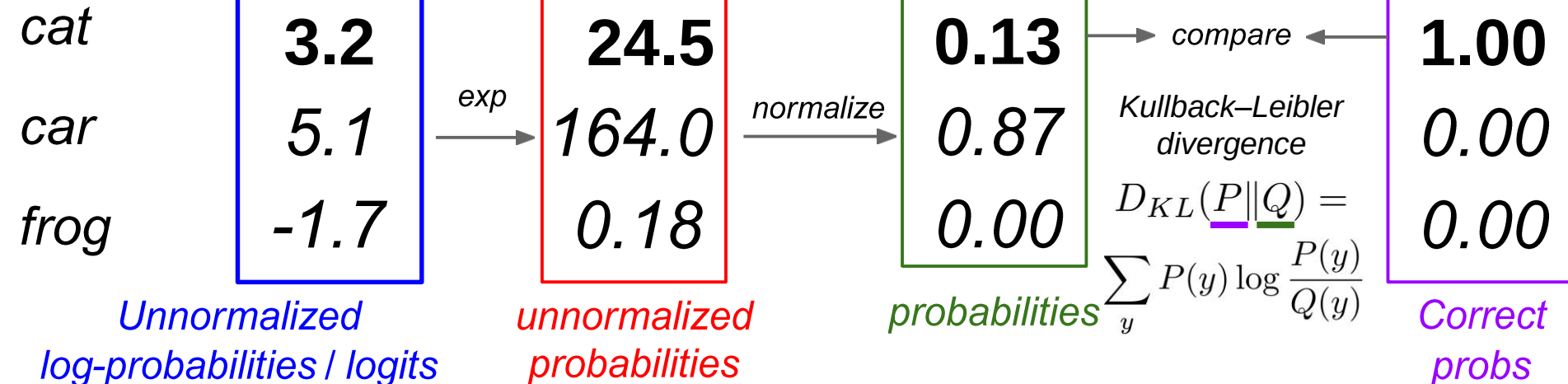
$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

*Probabilities
must be ≥ 0*

*Probabilities
must sum to 1*

$$L_i = -\log P(Y = y_i|X = x_i)$$



Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



$$s = f(x_i; W)$$

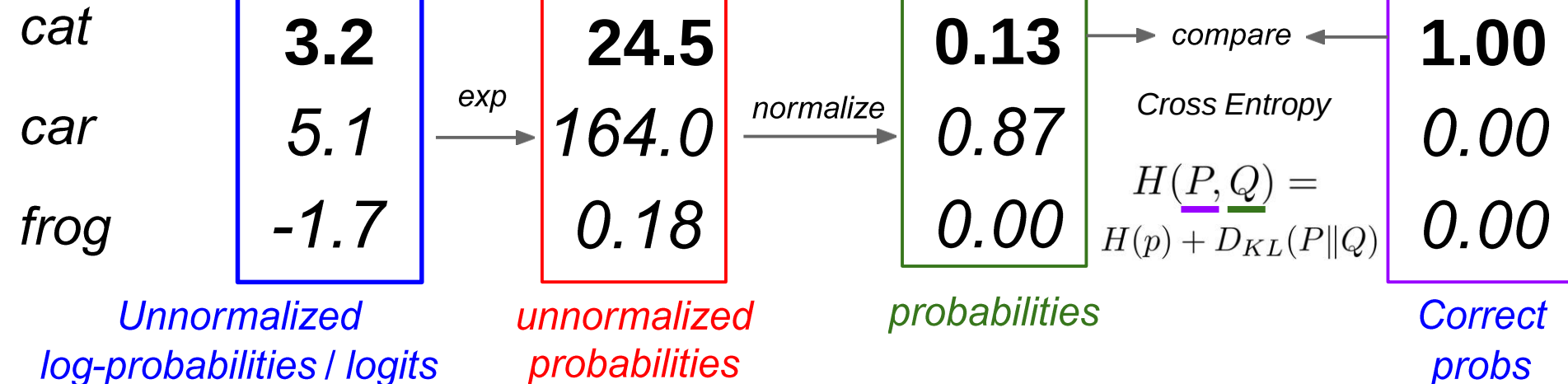
$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

*Probabilities
must be ≥ 0*

*Probabilities
must sum to 1*

$$L_i = -\log P(Y = y_i|X = x_i)$$



Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Maximize probability of correct class

$$L_i = -\log P(Y = y_i | X = x_i)$$

Putting it all together:

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

cat	3.2
car	5.1
frog	-1.7

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Maximize probability of correct class

$$L_i = -\log P(Y = y_i | X = x_i)$$

Putting it all together:

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

cat	3.2
car	5.1
frog	-1.7

Q1: What is the min/max possible softmax loss L_i ?

Q2: At initialization all s_j will be approximately equal; what is the softmax loss L_i , assuming C classes?



Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Maximize probability of correct class

$$L_i = -\log P(Y = y_i | X = x_i)$$

Putting it all together:

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

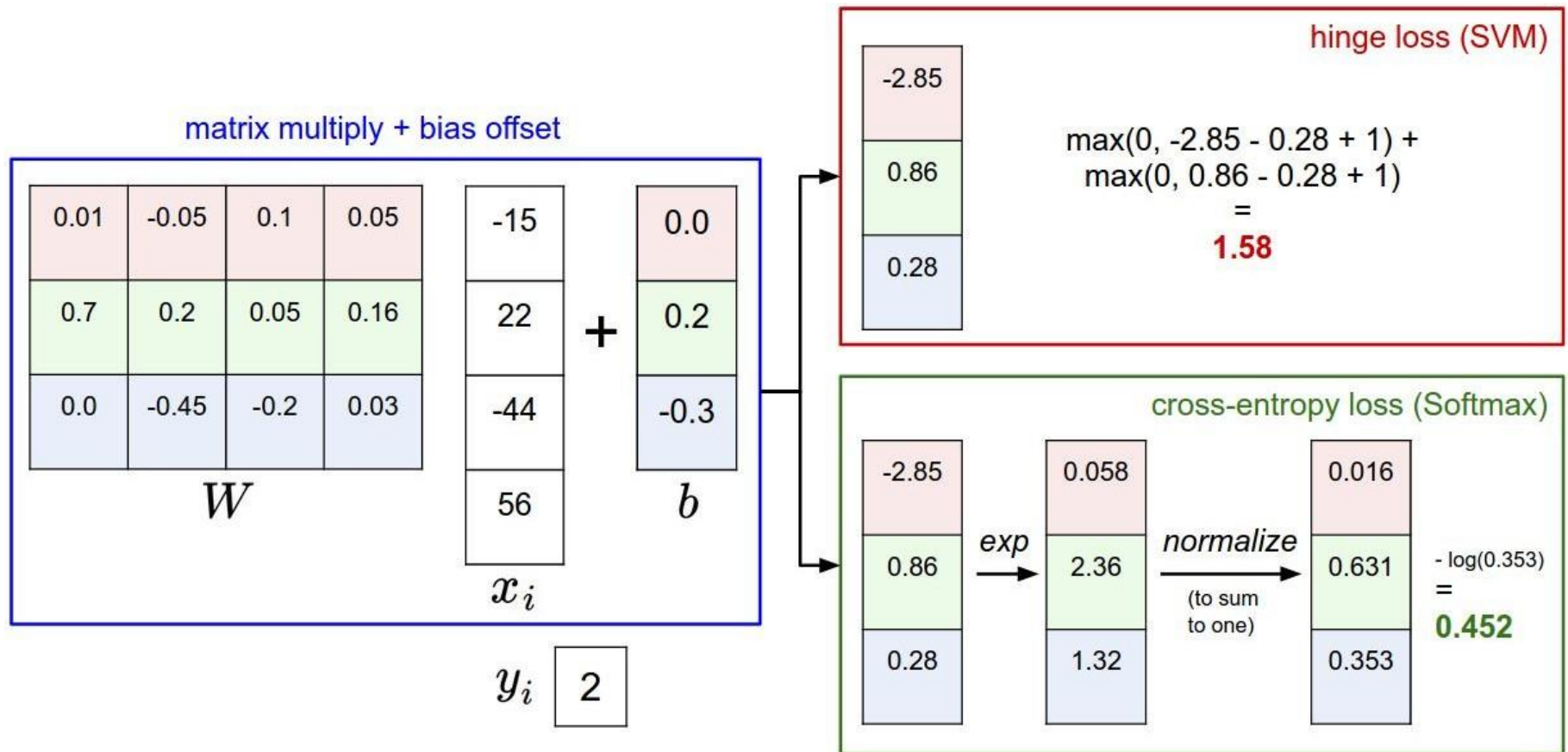
cat	3.2
car	5.1
frog	-1.7

Q2: At initialization all s will be approximately equal;
what is the loss?

A: $-\log(1/C) = \log(C)$,

If $C = 10$, then $L_i = \log(10) \approx 2.3$

Softmax vs. SVM



Softmax vs. SVM

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Softmax vs. SVM

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

assume scores:

[10, -2, 3]

[10, 9, 9]

[10, -100, -100]

and $y_i = 0$

*Q: What is the **softmax loss** and the **SVM loss**?*

Softmax vs. SVM

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

assume scores:

[20, -2, 3]

[20, 9, 9]

[20, -100, -100]

and $y_i = 0$

*Q: What is the **softmax loss** and the **SVM loss** if I double the correct class score from 10 -> 20?*



感知机





感知机

Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN¹

F. ROSENBLATT

Cornell Aeronautical Laboratory

HAVING told you about the giant digital computer known as **I.B.M. 704** and how it has been taught to play a fairly creditable game of chess, we'd like to tell you about an even more remarkable machine, the perceptron, which, as its name implies, is capable of what amounts to original thought. The first perceptron has yet to be built,

The New Yorker, December 6, 1958 P. 44



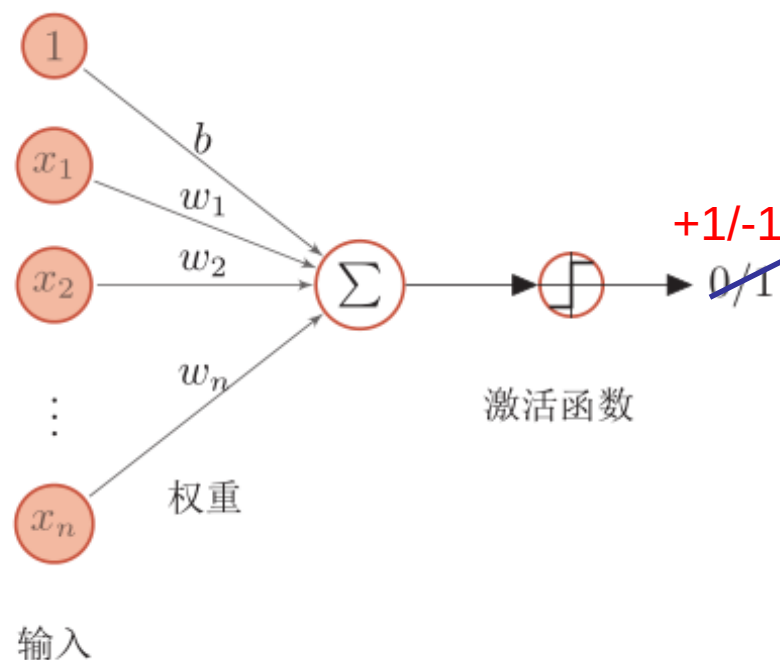
The IBM 704 computer



感知机

- 模拟生物神经元行为的机器，有与生物神经元相对应的部件，如权重（突触）、偏置（阈值）及激活函数（细胞体），输出为+1或-1。

$$\hat{y} = \begin{cases} +1 & \text{当 } \mathbf{w}^T \mathbf{x} > 0 \\ -1 & \text{当 } \mathbf{w}^T \mathbf{x} \leq 0 \end{cases},$$





感知机

- 学习算法

- 一种错误驱动的在线学习算法：
- 先初始化一个权重向量 $\mathbf{w} \leftarrow \mathbf{0}$ （通常是全零向量）；
- 每次分错一个样本 $(\mathbf{x}, y)$ 时，即

$$y\mathbf{w}^T \mathbf{x} < 0$$

- 用这个样本来更新权重

$$\mathbf{w} \leftarrow \mathbf{w} + y\mathbf{x}$$

- 根据感知机的学习策略，可以反推出感知机的损失函数为

$$\mathcal{L}(\mathbf{w}; \mathbf{x}, y) = \max(0, -y\mathbf{w}^T \mathbf{x})$$





感知机的学习过程

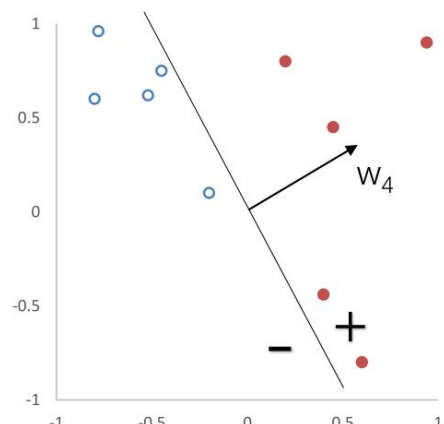
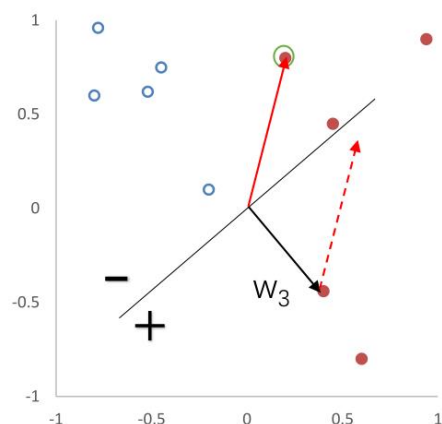
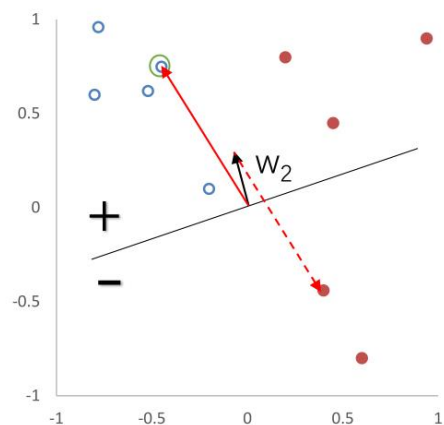
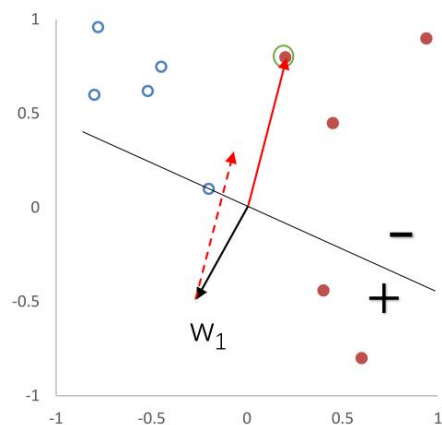
算法 3.1 两类感知器的参数学习算法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 最大迭代次数 T

```
1 初始化:  $\mathbf{w}_0 \leftarrow 0, k \leftarrow 0, t \leftarrow 0$ ;  
2 repeat  
3   对训练集  $\mathcal{D}$  中的样本随机排序;  
4   for  $n = 1 \cdots N$  do  
5     选取一个样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;  
6     if  $\mathbf{w}_k^\top (y^{(n)} \mathbf{x}^{(n)}) \leq 0$  then 表示分错  
7        $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k + y^{(n)} \mathbf{x}^{(n)}$ ;  
8        $k \leftarrow k + 1$ ;  
9     end  
10     $t \leftarrow t + 1$ ;  
11    if  $t = T$  then break; // 达到最大迭代次数  
12  end  
13 until  $t = T$ ;  
   输出:  $\mathbf{w}_k$ 
```



感知机参数学习的更新过程





收敛性

当数据集是两类线性可分时,对于训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 其中 $\mathbf{x}^{(n)}$ 为样本的增广特征向量, $y^{(n)} \in \{-1, 1\}$, 那么存在一个正的常数 $\gamma (\gamma > 0)$ 和权重向量 $\mathbf{w}^*$, 并且 $\|\mathbf{w}^*\| = 1$, 对所有 n 都满足 $(\mathbf{w}^*)^\top (y^{(n)} \mathbf{x}^{(n)}) \geq \gamma$. 我们可以证明如下定理.

定理 3.1 – 感知器收敛性: 给定训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 令 R 是训练集中最大的特征向量的模, 即

$$R = \max_n \|\mathbf{x}^{(n)}\|.$$

如果训练集 $\mathcal{D}$ 线性可分, 两类感知器的参数学习算法 3.1 的权重更新次数不超过 $\frac{R^2}{\gamma^2}$.





证明过程

- 感知机的权重向量的更新方式为

$$\mathbf{w}_k = \mathbf{w}_{k-1} + y^{(k)} \mathbf{x}^{(k)}$$

其中 $\mathbf{x}^{(k)}, y^{(k)}$ 表示第 k 个错误分类的样本.

- 在第 K 次更新时感知机的权重向量为

$$\mathbf{w}_K = \sum_{k=1}^K y^{(k)} \mathbf{x}^{(k)}$$

- $\|\mathbf{w}_K\|^2$ 的上界
- $\|\mathbf{w}_K\|^2$ 的下界

$$K^2 \gamma^2 \leq \|\mathbf{w}_K\|^2 \leq KR^2.$$





线性分类模型小结

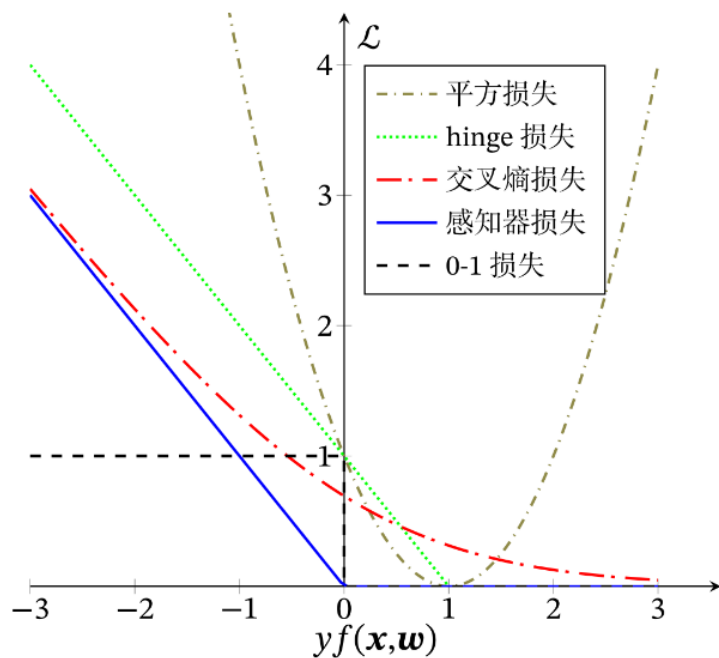
线性模型	激活函数	损失函数	优化方法
线性回归	-	$(y - \mathbf{w}^\top \mathbf{x})^2$	最小二乘、梯度下降
Logistic 回归	$\sigma(\mathbf{w}^\top \mathbf{x})$	$\mathbf{y} \log \sigma(\mathbf{w}^\top \mathbf{x})$	梯度下降
Softmax 回归	$\text{softmax}(\mathbf{W}^\top \mathbf{x})$	$\mathbf{y} \log \text{softmax}(\mathbf{W}^\top \mathbf{x})$	梯度下降
感知器	$\text{sgn}(\mathbf{w}^\top \mathbf{x})$	$\max(0, -y\mathbf{w}^\top \mathbf{x})$	随机梯度下降
支持向量机	$\text{sgn}(\mathbf{w}^\top \mathbf{x})$	$\max(0, 1 - y\mathbf{w}^\top \mathbf{x})$	二次规划、SMO 等





不同损失函数的对比

为了比较这些损失函数，我们统一定义类别标签 $y \in \{+1, -1\}$ ，并定义 $f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x} + b$ 。这样对于样本 $(\mathbf{x}, y)$ ，若 $yf(\mathbf{x}; \mathbf{w}) > 0$ ，则分类正确，相反则分类错误。这样为了方便比较这些模型，我们可以将它们的损失函数都表述为定义在 $yf(\mathbf{x}; \mathbf{w})$ 上的函数。



$$\mathcal{L}_{LR} = \log(1 + \exp(-yf(\mathbf{x}; \mathbf{w})))$$

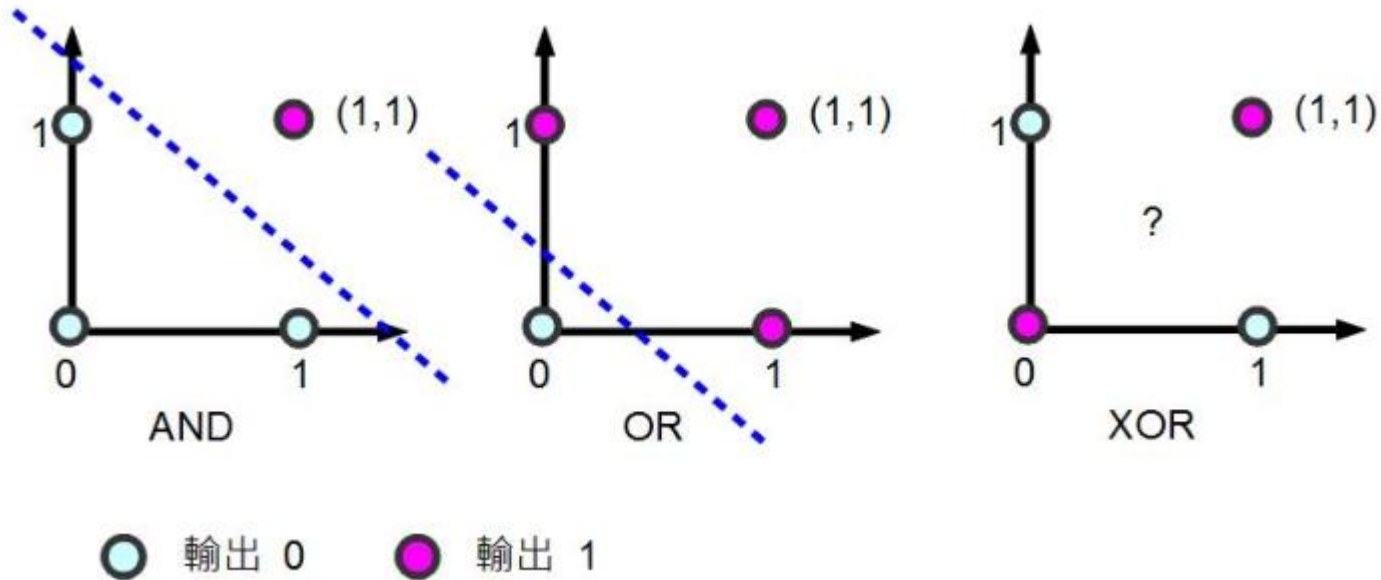
$$\mathcal{L}_{hinge} = \max(0, 1 - yf(\mathbf{x}; \mathbf{w}))$$

$$\mathcal{L}_p = \max(0, -yf(\mathbf{x}; \mathbf{w}))$$

$$\mathcal{L}_{squared} = (1 - yf(\mathbf{x}; \mathbf{w}))^2$$



XOR问题



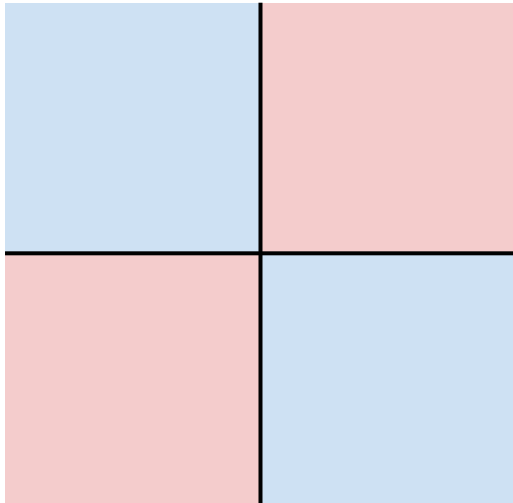
Hard cases for a linear classifier

Class 1:

First and third quadrants

Class 2:

Second and fourth quadrants

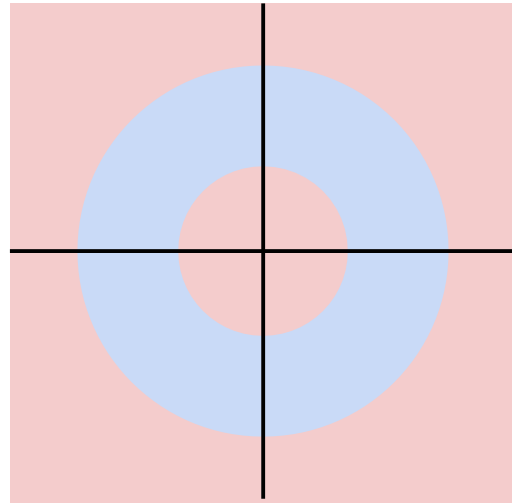


Class 1:

$1 \leq L2 \text{ norm} \leq 2$

Class 2:

Everything else



Class 1:

Three modes

Class 2:

Everything else

